PROGARAM

```
public class QuickSortNames {
    public static void main(String[] args) {
        String[] names = { "John", "Alice", "Bob", "Eve", "Charlie", "David", "Frank" };
        System.out.println("Original List:");
        printArray(names);
        quickSort(names, 0, names.length - 1);
        System.out.println("\nSorted List:");
        printArray(names);
    }
    // Quick Sort algorithm
    public static void quickSort(String[] arr, int low, int high) {
        if (low < high) {
            int partitionIndex = partition(arr, low, high);
            // Recursively sort elements before and after the partition
            quickSort(arr, low, partitionIndex - 1);
            quickSort(arr, partitionIndex + 1, high);
        }
    }
    // Partitioning method to find the correct position of the pivot element
    public static int partition(String[] arr, int low, int high) {
        String pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j].compareTo(pivot) <= 0) {
                i++;
                // Swap arr[i] and arr[j]
                String temp = arr[i];
```

```
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        // Swap arr[i+1] and arr[high] (pivot)
        String temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }
    // Utility method to print an array
    public static void printArray(String[] arr) {
        for (String name : arr) {
            System.out.print(name + " ");
        }
        System.out.println();
    }
}
```

OUTPUT

```
[Running] cd "c:\Users\HP\Desktop\LAB\" && javac QuickSortNames.java && java QuickSortNames
Original List:
John Alice Bob Eve Charlie David Frank

Sorted List:
Alice Bob Charlie David Eve Frank John

[Done] exited with code=0 in 0.941 seconds
```

ALGORITHM

1.Initialize an array

- Define an array of names
- Print the original list of names

2.Call the quickSort method with the array, starting index 0 and ending index length - 1

3.In partition method, select the last element as pivot

- Loop through the array
- If an element is less than or equal to the pivot, swap it to the left side
- Place the pivot at its correct sorted position by swapping it with the element at index i+1
- Return the index of the pivot

4.Recursive sorting:

- In quicksort, recursively sort the elements to the left of the pivot and to the right of the pivot until the whole array is sorted.

5.Print the sorted array:

- After sorting, print the final sorted array of names